

Advanced Cognitive Architectures: Leveraging the Open Knowledge Format for Progressive Disclosure, Semantic Routing, and Context Optimization

The rapid maturation of artificial intelligence has fundamentally shifted the engineering bottleneck from the internal parameter weights of large language models (LLMs) to the external orchestration of their runtime environments. Early deployments of generative AI relied almost entirely on monolithic context windows, an architecture wherein all requisite instructions, historical state, and tool schemas were forcefully injected into a single prompt payload¹. While this approach is sufficient for isolated, stateless queries, it suffers from catastrophic degradation when applied to long-horizon, multi-step agentic workflows. As context length expands, models inevitably encounter attention scarcity, the "lost-in-the-middle" phenomenon, and severe U-shaped attention curve decay, leading to hallucinations and the dilution of critical procedural instructions¹.

To circumvent the strict upper bounds of token limits and attention mechanics, the field of cognitive architecture has pivoted toward memory externalization¹. By treating the LLM not as a holistic mind but as a central processing unit (CPU) that reads from and writes to an external filesystem, engineers can construct persistent, compounding memory structures¹. This architectural evolution is currently being standardized through the Open Knowledge Format (OKF), a vendor-neutral specification designed to represent curated knowledge across agentic systems⁵. This report provides an exhaustive analysis of OKF and its integration into modern AI mind architectures, detailing the implementation of progressive disclosure, the mechanics of YAML-driven semantic routing, and the codification of procedural memory.

The Epistemological Crisis of Stateless AI and the LLM Wiki Paradigm

Before analyzing the formalized OKF specification, it is imperative to understand the structural failures of traditional Retrieval-Augmented Generation (RAG) that necessitated its creation. RAG systems typically operate on a stateless paradigm: upon receiving a user query, the system vectorizes the prompt, retrieves statistically similar text chunks from a database, and injects them into the model's context window⁷. While effective for pinpoint factual retrieval, RAG fundamentally fails at complex knowledge synthesis because nothing accumulates between sessions⁴. The agent must repeatedly rediscover the topological relationships of the data from scratch, squandering finite computational resources and token budgets⁴. In response to this systemic inefficiency, the AI research community, heavily influenced by

frameworks introduced by industry leaders such as Andrej Karpathy, developed the "LLM Wiki" pattern⁴. This paradigm reimagines knowledge management by positioning the AI agent as an active, continuous curator of a persistent filesystem⁷. In this architecture, raw input documents—such as research papers, meeting transcripts, or API logs—are treated as immutable source code³. The agent acts as a compiler, reading the raw sources and synthesizing them into a dense, interconnected web of Markdown files³.

When a subsequent query is issued, the agent does not parse the noisy raw data; rather, it navigates the pre-compiled, highly structured wiki³. This approach transforms disjointed information into a compounding artifact, significantly reducing the cognitive load and token consumption required for the agent to orient itself within a complex domain⁴. The success of the LLM Wiki pattern proved that agents excel when they are provided with an integrated, spatial layout of information, yet the proliferation of bespoke, ad-hoc wiki structures created a new challenge: knowledge islands that could not interoperate across different agent frameworks⁹.

The Open Knowledge Format (OKF) Specification

To resolve the fragmentation of metadata and the isolation of individual LLM wikis, the Google Cloud Data Cloud team introduced the Open Knowledge Format (OKF) v0.1 on June 12, 2026⁵. OKF is an open, vendor-neutral standard that formalizes the LLM wiki pattern into a highly portable, interoperable specification⁵. By utilizing standard UTF-8 Markdown files augmented with YAML frontmatter, OKF ensures that organizational knowledge remains entirely readable by human operators while being strictly parseable by autonomous agents⁵.

The defining architectural decision of OKF is its absolute rejection of proprietary runtimes, specific cloud platforms, and required software development kits (SDKs)⁵. An OKF knowledge repository—termed a "bundle" or "forest"—is fundamentally just a directory tree residing on a standard filesystem or version control repository, ensuring absolute data sovereignty and eliminating vendor lock-in⁶.

Structural Anatomy of an OKF Concept

The fundamental atomic unit of knowledge within an OKF bundle is the "concept" (or "tree")¹⁴. A concept can represent a tangible physical asset, such as a database table or a server architecture, or an abstract organizational idea, such as a business metric or a standard operating procedure⁶. The deterministic identity of any concept is established strictly by its file path within the bundle, effectively utilizing the filesystem hierarchy as the primary ontological structure⁶.

The specification strictly mandates that every concept document must be bifurcated into two distinct computational zones: the structured YAML frontmatter and the unstructured Markdown body⁶. This separation intentionally mixes queryable, machine-readable metadata with the rich, narrative prose required for LLM reasoning¹⁵.

The format is deliberately under-specified, adhering to a principle of minimal interference. The specification defines the interoperability surface without dictating the underlying content

model⁵. This allows diverse teams within an organization to maintain bespoke taxonomies while coexisting within a unified agentic context window¹⁴.

OKF Frontmatter Field	Specification Status	Architectural Function within the Agent Runtime
type	Strictly Required	Defines the foundational ontological category (e.g., Metric, Table, API). Dictates how semantic routers classify the document.
title	Recommended	Human-readable display name. If omitted, compliant consumers derive the title dynamically from the filesystem path.
description	Recommended	A dense, single-sentence summary. Serves as the primary mechanism for progressive disclosure and semantic routing algorithms.
resource	Recommended	A URI pointing to the underlying asset being described, establishing a bridge between semantic knowledge and physical infrastructure.
tags	Recommended	An array of strings used for cross-cutting categorical indexing, allowing agents to perform localized grep or search operations.
timestamp	Recommended	ISO 8601 datetime indicating the last meaningful semantic

		mutation. Vital for agents evaluating the staleness of procedural data.
--	--	---

Beneath the YAML frontmatter lies the Markdown body. Producers are highly encouraged to prioritize structural Markdown elements—such as deeply nested headings, definition lists, and fenced code blocks—over unstructured freeform prose, as structural markers significantly enhance the spatial navigation capabilities of parsing agents⁶.

Crucially, OKF concepts establish relationships through standard Markdown hyperlinks¹⁵. These links transform the hierarchical directory tree into a multidimensional, directed knowledge graph⁶. The specific semantics of the relationship (e.g., a "depends-on" execution constraint or a "joins-with" data architecture) are articulated within the surrounding narrative prose rather than the link itself¹⁶. Compliant OKF consumer agents must tolerate broken cross-links, treating them not as fatal errors but as pointers to not-yet-synthesized knowledge⁶.

Reference Implementations and Ecosystem Interoperability

To demonstrate the viability of the format, Google Cloud released several reference implementations alongside the v0.1 specification¹⁸. The primary producer example is a BigQuery enrichment agent⁵. This autonomous agent recursively crawls a BigQuery dataset, drafting a distinct OKF concept document for every identified table and view⁵. It then executes a secondary processing pass, crawling disparate authoritative documentation sources to enrich the newly generated concepts with complex join paths, detailed schemas, and rigorously formatted citations⁵.

On the consumer side, the reference implementations include a static HTML visualizer that renders any local OKF bundle into an interactive, self-contained graph network without requiring backend server processing, as well as native ingestion support within Google Cloud's Knowledge Catalog⁵. However, because the format is strictly standard Markdown and YAML, the ecosystem extends far beyond these reference tools. Third-party visualizers, native IDE extensions, and personal knowledge management software such as Obsidian natively support the editing, traversal, and rendering of OKF bundles, rendering the format universally accessible⁵.

Information Theory and the Mechanics of Progressive Disclosure

The most profound constraint in modern AI engineering is the hard limit on channel capacity within the context window²⁵. When an agent is forced to process an entire repository of knowledge simultaneously, the signal-to-noise ratio collapses². Progressive disclosure, a concept originally pioneered in user interface design to minimize human cognitive load, has

been repurposed as the foundational engineering philosophy for context optimization²⁵. The objective is to deliver the absolute minimum viable semantic information into the context window at precisely the moment it is required¹².

Architectural Integration of Navigational Indexes

The OKF specification directly operationalizes progressive disclosure through the use of a reserved filename: index.md¹⁴. An index.md file may be placed at any directory level within an OKF bundle, serving as a localized table of contents¹⁶. When an autonomous agent begins exploring a novel knowledge domain, it does not execute a bulk read of the directory contents; rather, it loads only the index.md file²⁰.

This index file enumerates the concepts and subdirectories available within its immediate scope, typically appending the dense, one-sentence description extracted from each concept's YAML frontmatter¹⁶. By processing this highly compressed index—which rarely consumes more than a few hundred tokens—the agent constructs an internal map of the domain's topography¹⁷. Utilizing this map, the agent makes deterministic routing decisions, traversing deeper into the hierarchy and opening only the specific concept files mathematically required to fulfill its current objective²⁰.

Mitigating Context Crowding via the Artifact Pyramid

While the flat, peer-to-peer structure of standard OKF is highly flexible, it can induce context crowding during complex, multi-agent workflows. When high-level strategic playbooks are stored at the same filesystem depth as granular database schemas, an orchestrator agent searching for architectural guidance is forced to expend context capacity evaluating irrelevant technical specifications²⁸.

To optimize progressive disclosure at scale, OKF bundles can integrate the "Artifact Pyramid" methodology²⁸. This design pattern enforces a rigorous vertical stratification of knowledge, segregating concepts into three distinct ontological layers based on their degree of synthesis²⁸.

Knowledge Stratum	Composition and Content	Target Agent Consumer Profile
Layer 1 (L1) - Synthesis	High-level executive summaries, primary research findings, and strategic implications consolidated into a single artifact.	Orchestrator agents, meta-planners, and executive routing functions requiring rapid orientation.
Layer 2 (L2) - Analysis	Deeply focused, domain-specific investigations (e.g.,	Specialized worker subagents executing narrow, bounded tasks

	competitive market mapping, technical latency benchmarks).	within a specific discipline.
Layer 3 (L3) - Raw Dossiers	Unaltered source transcripts, raw statistical data tables, and foundational methodology notes.	Validator and auditing agents responsible for verifying provenance and hallucination detection.

The critical innovation of the Artifact Pyramid is its mechanism for zero-cost context prediction²⁸. Within this methodology, every concept file must include a highly structured SOURCES section appended to the Markdown body²⁸. Each entry within the SOURCES block pairs a standard Markdown link with a mandatory, single-line description of the target file's contents (e.g., research/analysis/market-position.md -> Competitor mapping and market share analysis supporting Section 2)²⁸.

This implementation represents the apotheosis of progressive disclosure. When an agent reads an L1 synthesis file, it evaluates the annotated SOURCES hints to determine if navigating to an L2 analysis file is necessary²⁸. The context cost of this relevance decision is absolute zero, as the descriptive hint is embedded within the file the agent has already loaded²⁸. The agent is never forced to execute a filesystem read operation merely to discover a document's contents²⁸. Furthermore, this strict hierarchical linking ensures audit-proof provenance by construction, as every synthetic assertion in an L1 document algorithmically traces down through the L2 analyses to the foundational L3 evidence²⁸.

YAML Metadata Semantic Routing at the Edge

While progressive disclosure optimizes filesystem traversal, the initial orchestration of tasks requires highly sophisticated request routing. In a modern cognitive architecture, a single user prompt may need to be handled by a massive frontier reasoning model, a heavily quantized local coding assistant, or a specialized multi-agent sub-routine²⁹. Traditional routing mechanisms rely on static keyword matching or rigid regular expressions, which are incredibly brittle and universally fail to capture the nuanced semantic intent of natural language instructions²⁹.

Semantic routing resolves this vulnerability by vectorizing the incoming user prompt and computing its cosine similarity against the embedded descriptions of available models, tools, and OKF concepts³⁰. The YAML frontmatter specified by OKF provides the exact metadata anchors required for these high-dimensional vector comparisons.

The Mechanics of the Semantic Router Control Plane

Production-grade semantic routing functions as a highly optimized, autonomous control plane that executes during the millisecond interval before a prompt reaches a generative inference engine³³. Architectures such as the vLLM Semantic Router (specifically the Themis v0.3 release) deploy this logic at the very edge of the network, utilizing the Envoy External Processing (ExtProc) protocol to intercept and mutate requests directly within the API gateway layer, typically over port 50051³³.

To eliminate the latency penalty associated with external API calls, these routers utilize embedded, highly efficient embedding models. For example, a 2D Matryoshka mmBERT embedding model, consuming approximately 130MB of memory, can execute dense vector classifications directly on the CPU in a single millisecond²⁹. The request lifecycle within the semantic router traverses a rigorous sequence of computational layers³³.

Routing Layer	Computational Function within the Request Lifecycle
Signal Extraction	Analyzes the raw request to extract evidence using embedding similarity, PII classifiers, urgency scoring, and keyword detection.
Policy Projection	Normalizes the extracted signals into composite, policy-ready concepts (e.g., calculating an aggregate "verification required" or "clinical intent" score based on weighted thresholds).
Decision Logic	Evaluates the normalized projections against priority-ordered, non-Turing-complete decision trees to isolate the appropriate execution path.
Algorithm Selection	Chooses the optimal backend candidate within the matched decision branch utilizing one of 14 registered algorithms, including Multi-Factor weighting, ELO ranking, or Reinforcement Learning (RL) driven policies.
Model Dispatch	Mutates the original API request, dynamically rewriting the target model endpoint before forwarding the payload to the inference engine.

Semantic Anchoring via YAML Configuration

Within this architecture, the routing destinations are defined through declarative YAML configuration files²⁹. Each available asset—whether it is an LLM, a specific OKF knowledge bundle, or a procedural tool—is registered with a natural language description parameter²⁹. The central insight of semantic routing is that these natural language descriptions serve as the mathematical anchors for the embedding space²⁹.

If a routing configuration contains an entry such as description: Specialized coding model optimized for programming tasks, debugging, algorithms, and technical implementation in Python and a user submits a prompt stating "Analyze the time complexity of this sorting sequence," the router vectorizes the prompt²⁹. Because the semantic meaning of the prompt occupies a similar geometric region in the high-dimensional vector space as the YAML description, the router calculates a high cosine similarity score³⁰. Provided this score exceeds the predefined confidence threshold, the router transparently mutates the request payload and forwards it to the specialized coding model²⁹. This entire decision matrix operates on pure semantics, ensuring that complex, edge-case queries are consistently routed to the optimal processing node without relying on fragile keyword matching³⁰.

The Codification of Procedural Memory

While OKF bundles excel at structuring semantic memory (durable facts, architectural relationships, and datasets) and episodic memory (chronological histories tracked via log.md), fully autonomous AI systems require a robust framework for procedural memory¹. Procedural memory dictates *how* an agent should behave, defining the boundaries of its autonomy, its error-handling heuristics, and the specific workflows it must follow to execute complex tasks¹. Within modern file-backed cognitive architectures, this procedural knowledge is codified via the AGENTS.md and SKILL.md specifications³⁷. Much like OKF concepts, these artifacts are Markdown files equipped with YAML frontmatter, ensuring they are tracked in version control, subjected to peer review, and deployed directly alongside the source code they govern³⁷.

AGENTS.md: The Operational Contract

The AGENTS.md file serves as the unified operational contract for a repository, mitigating the environmental ambiguity that frequently causes agentic failures³⁹. Historically, AI instructions were fragmented across tool-specific configuration files or buried within ephemeral chat histories, leading to inconsistent behaviors when different developers interacted with the same codebase using different agent frameworks³⁹. The AGENTS.md specification—now adopted by over 60,000 projects and stewarded by the Agentic AI Foundation under the Linux Foundation—standardizes this context layer, providing universal compatibility across platforms such as Codex, Cursor, and Copilot³⁸.

To maximize token efficiency and ensure deterministic execution, the AGENTS.md file must strictly adhere to specific prompt engineering patterns:

- **Command-First Architecture:** Autonomous agents do not benefit from human-readable prose or vague directives such as "be careful"³⁸. Every instruction must map to an exact, executable terminal invocation. Agents must be able to verify the success or failure of their actions by analyzing process exit codes³⁸.
- **Byte-Capped Executions:** To maintain strict context discipline, agents instructed by AGENTS.md must utilize byte-capped command outputs for exploratory operations (e.g., executing `COMMAND 2>&1 | head -c 4000`)⁴¹. This prevents massive log files or unstructured outputs from flooding the context window, which would otherwise displace critical reasoning tokens and degrade output quality⁴¹.
- **Closure Definitions:** The file must explicitly define the exact parameters of task completion, preventing premature termination. A definition of done must rely on specific metrics, such as passing test suites or zero linter warnings³⁸.
- **Escalation and Defensive Constraints:** When blocked, autonomous agents tend to improvise, frequently resorting to destructive workarounds such as deleting lock files or bypassing security checks³⁸. AGENTS.md must define strict escalation paths, explicit "ask-first" permission boundaries, and absolute prohibitions against destructive actions, creating a robust safety net for autonomous operation³⁸.

Dynamic Capability Extension via SKILL.md

When agentic workflows require highly specialized, multi-step orchestration that exceeds the general scope of a repository's AGENTS.md file, the architecture relies on Agent Skills¹². A skill is a self-contained, filesystem-based package that equips an agent with on-demand procedural expertise, resolving the tension between the broad knowledge of a generalized model and the specific requirements of a domain task¹².

Structured within a dedicated directory, a skill is defined by its SKILL.md file¹². This file enforces rigorous YAML frontmatter constraints, mandating a unique name (restricted to lowercase alphanumeric characters and hyphens, matching the parent directory) and a detailed description³⁷. The description field is particularly critical; it is not intended for human consumption but serves as the explicit semantic trigger condition for the agent's routing algorithms⁴⁴.

The execution of a skill represents the highest tier of progressive disclosure, operating through a highly optimized, three-stage loading protocol designed to minimize context window consumption:

1. **Stage 1 - Advertise:** During the initialization phase, the agent executes a filesystem scan of all installed skills. It reads *only* the YAML frontmatter (the name and description) from each SKILL.md file⁴⁴. This metadata extraction consumes roughly 100 tokens per skill, allowing the agent to maintain an extensive index of available capabilities in its system prompt without exhausting its attention budget⁴⁴.
2. **Stage 2 - Load:** When a user's prompt semantically aligns with a skill's advertised description, the agent executes a tool call to load that specific skill. Only at this juncture does the agent read the full Markdown body of the SKILL.md file into its active context

window, absorbing the highly detailed standard operating procedures (SOPs), edge-case logic, and validation requirements³⁷.

3. **Stage 3 - Retrieve Resources:** If the skill's instructions reference supplementary data—such as architectural templates in an assets/ directory, lengthy regulatory documents in a references/ directory, or executable validation code in a scripts/ directory—the agent retrieves these files on demand via recursive read operations⁴⁴. This final tier guarantees that dense, high-token reference material is only materialized when mathematically required by the ongoing workflow⁴⁷.

By offloading complex procedural memory to the filesystem as executable SOPs, agents can overcome the "Complexity Ceiling" that typically limits long-horizon tasks, enabling autonomous execution, parallelized task decomposition, and deterministic self-correction loops⁴³.

Topologies of Multi-Agent Orchestration

The integration of semantic memory (OKF), episodic memory (log.md), and procedural memory (SKILL.md and AGENTS.md) establishes a comprehensive cognitive architecture capable of supporting sophisticated multi-agent orchestration¹. Modern AI systems rarely rely on a single monolithic model to handle all inputs; instead, they utilize the filesystem as a shared state mechanism to coordinate complex topologies¹.

Within frameworks such as LangChain and LangGraph, this orchestration manifests in several distinct architectural patterns³¹:

- **Subagents (Centralized Orchestration):** A lead orchestrator agent maintains a high-level topographical map of the OKF bundle via the root index.md files³¹. When presented with a complex objective, the lead agent decomposes the task and delegates specific components to specialized worker subagents³¹. The orchestrator passes only the precise filesystem paths and localized context required for each worker, enforcing context isolation and preventing token overflow³¹.
- **Handoffs (State-Driven Transitions):** In workflows requiring sequential domain expertise, control is transferred between distinct specialist agents based on the evolving state of the task³¹. When a new agent assumes control, it initializes its context by reading the episodic log.md file, seamlessly synchronizing its internal state with the progress and decisions logged by the preceding agent³¹.
- **Parallel Routers:** Utilizing the YAML-driven semantic router at the API gateway, a highly complex query can be dispatched simultaneously to multiple, disparate agent instances²⁹. Each agent executes its portion of the workflow, and a final synthesizer node aggregates their structured outputs into a cohesive response³¹.

Throughout all of these topological variations, the underlying state remains tethered to the filesystem. Because formats like OKF and SKILL.md define interoperability rather than enforcing a proprietary platform, an architectural document synthesized by a data enrichment agent can be immediately consumed and operationalized by a downstream coding agent, completely bypassing the need for complex API integrations or data translation layers⁵. By

utilizing standard files as the medium for cognitive externalization, these architectures achieve a level of resilience, transparency, and scalability that internal parameter tuning alone cannot provide¹.

Works cited

1. Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering - arXiv, <https://arxiv.org/html/2604.08224v1>
2. muratcankoylan/Agent-Skills-for-Context-Engineering - GitHub, <https://github.com/muratcankoylan/agent-skills-for-context-engineering>
3. Andrej Karpathy's LLM Wiki: Create your own knowledge base | by Urvil Joshi | Medium, <https://medium.com/@urvvil08/andrej-karpathys-llm-wiki-create-your-own-knowledge-base-8779014accd5>
4. Karpathy's LLM Wiki: A Knowledge Base That Compounds - AI Builder Club, <https://www.aibuilderclub.com/blog/karpathy-llm-wiki>
5. How the Open Knowledge Format can improve data sharing | Google Cloud Blog, <https://cloud.google.com/blog/products/data-analytics/how-the-open-knowledge-format-can-improve-data-sharing>
6. Open Knowledge Format | Definition, Scope and Compliance (Grounding Page), <https://groundingpage.com/facts/open-knowledge-format/>
7. LLM Wiki by Andrej Karpathy: Build a Compounding Knowledge Base (Tutorial), <https://datasciencedojo.com/blog/llm-wiki-tutorial/>
8. Wiki Memory - LangChain, <https://www.langchain.com/blog/wiki-memory>
9. Open Knowledge Format OKF: Turning Docs into Agent Memory - Document360, <https://document360.com/blog/open-knowledge-format/>
10. LLM Wiki - GitHub Gist, <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>
11. Karpathy's LLM Wiki: build/query interlinked markdown KB | Hermes Agent, <https://hermes-agent.nousresearch.com/docs/user-guide/skills/bundled/research/research-llm-wiki>
12. Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward - OpenReview, <https://openreview.net/pdf?id=ErOp92BsmW>
13. Software as Content: Dynamic Applications as the Human-Agent Interaction Layer - arXiv, <https://arxiv.org/html/2603.21334v1>
14. Google's Open Knowledge Format: The Markdown Standard That Could Replace Your Wiki, <https://flowtivity.ai/blog/google-open-knowledge-format/>
15. Google's New Knowledge Standard: What is the Open Knowledge Format (OKF)? | by Dreamwalker(박제창) | Jun, 2026 | Medium, <https://medium.com/@aristojeff/googles-new-knowledge-standard-what-is-the-open-knowledge-format-okf-f044ddf5b6bd>
16. knowledge-catalog/okf/SPEC.md at main - GitHub, <https://github.com/GoogleCloudPlatform/knowledge-catalog/blob/main/okf/SPEC.md>
17. Open Knowledge Format (OKF): The Open Standard That Frees Your AI

- Knowledge From Silos - innFactory AI Consulting,
<https://innfactory.ai/en/blog/open-knowledge-format-okf-standard-for-ai-knowledge/>
18. Open Knowledge Format (OKF): How Brands Can Structure Knowledge for AI Agents - Vizup,
<https://www.tryvizup.com/blog/open-knowledge-format-structuring-knowledge-for-ai-agents>
 19. knowledge-catalog/okf/README.md at main - GitHub,
<https://github.com/GoogleCloudPlatform/knowledge-catalog/blob/main/okf/README.md>
 20. Open Knowledge Format: I Already Do This, and Now It Has a Name | alexop.dev,
<https://alexop.dev/posts/open-knowledge-format-markdown-frontmatter-agent-knowledge/>
 21. Google's Open Knowledge Format Could Work For Websites, Too - No Hacks,
<https://nohacks.co/blog/okf-website-knowledge-graph>
 22. What is Open Knowledge Format(OKF)? | by Tahir | Jun, 2026 - Medium,
<https://medium.com/@tahirbalarabe2/what-is-open-knowledge-format-okf-270b20791802>
 23. Google Cloud Announces The Open Knowledge Format - Search Engine Journal,
<https://www.searchenginejournal.com/google-cloud-announces-the-open-knowledge-format/579253/>
 24. What Is Andrej Karpathy's LLM Wiki? How to Build a Personal Knowledge Base With Claude Code | MindStudio,
<https://www.mindstudio.ai/blog/andrej-karpathy-llm-wiki-knowledge-base-claude-code>
 25. Progressive Disclosure: The Core Engineering Philosophy of the LLM Era | by shuai zhang,
<https://medium.com/@dyzsasd/progressive-disclosure-the-core-engineering-philosophy-of-the-llm-era-0a6328774404>
 26. Where RAG Breaks Down: The Karpathy LLM Wiki Alternative - MindStudio,
<https://www.mindstudio.ai/blog/karpathy-llm-wiki-pattern-knowledge-base-without-rag>
 27. Open Knowledge Format: The New AI Infrastructure - Honra,
<https://www.honra.io/insights/open-knowledge-format-agent-ready-knowledge>
 28. How to Strengthen Google's OKF With a Methodology That Converged by Design,
<https://magnus919.com/2026/06/how-to-strengthen-googles-okf-with-a-methodology-that-converged-by-design/>
 29. Giving AgentGateway a Semantic Brain with vLLM Semantic Router,
<https://vllm-semantic-router.com/blog/agentgateway-semantic-brain-homelab/>
 30. Building a Production-Grade Semantic Router: The Smart Way to Route AI Prompts in 2026,
<https://atul4u.medium.com/building-a-production-grade-semantic-router-the-smart-way-to-route-ai-prompts-f303e6d2ae7e>
 31. AI agent frameworks & building blocks - AWS,

- <https://aws.amazon.com/marketplace/build-learn/ai-agent-learning-series/agent-frameworks-building-blocks>
32. Multi-LLM routing strategies for generative AI applications on AWS | Artificial Intelligence,
<https://aws.amazon.com/blogs/machine-learning/multi-llm-routing-strategies-for-generative-ai-applications-on-aws/>
 33. vLLM Semantic Router v0.3 Themis: From Signals to Stateful Production Routing,
<https://vllm.ai/blog/2026-06-05-v0.3-vllm-sr-themis-release>
 34. Semantic Routing Under the Microscope: Tracing Latency with OpenTelemetry and Coroot, <https://leminnov.blog/posts/semantic-routing-otel/>
 35. Karpathy's LLM Wiki as Agent Memory - Agentic AI Foundation (AAIF),
<https://aaif.io/blog/karpathys-llm-wiki-as-agent-memory/>
 36. Architecting Autonomous AI Systems: A Comprehensive Guide to Agents, Tool Calls, and Agent Skills - Parv Bhargava,
<https://bhargavaparv.medium.com/architecting-autonomous-ai-systems-a-comprehensive-guide-to-agents-tool-calls-and-agent-skills-731d5576d557>
 37. Agent Skills | Microsoft Learn,
<https://learn.microsoft.com/en-us/agent-framework/agents/skills>
 38. AGENTS.md Patterns: What Actually Changes Agent Behavior - Blake Crosley,
<https://blakecrosley.com/blog/agents-md-patterns>
 39. The Agent-Native Repo: Why AGENTS.MD is the New Standard | Harness Blog,
<https://www.harness.io/blog/the-agent-native-repo-why-agents-md-is-the-new-standard>
 40. AGENTS.md, <https://agents.md/>
 41. AGENTS.md patterns for context engineering in coding agents - GitHub,
<https://github.com/Austin1serb/agents-md>
 42. 5 AGENTS.md Patterns That 10x Your AI Coding Workflow (With Templates),
<https://dev.to/dohkoai/5-agentsmd-patterns-that-10x-your-ai-coding-workflow-with-templates-5ln>
 43. Coding Agent Skills: The Orchestration Framework for Industrial-Grade AI Workflows - GitHub, <https://github.com/you-iskw/coding-agent-skills>
 44. The SKILL.md Pattern: How to Write AI Agent Skills That Actually Work | by Bibek Poudel,
<https://bibek-poudel.medium.com/the-skill-md-pattern-how-to-write-ai-agent-skills-that-actually-work-72a3169dd7ee>
 45. Specification - Agent Skills, <https://agentskills.io/specification>
 46. Use Agent Skills in VS Code,
<https://code.visualstudio.com/docs/agent-customization/agent-skills>
 47. The SKILL.md Pattern: How to Write AI Agent Skills That Actually Work | by Sathish Raju,
<https://medium.com/@sathishkraju/the-skill-md-pattern-how-to-write-ai-agent-skills-that-actually-work-c4ab23400ed5>
 48. Cybernaut: Towards Reliable Web Automation - arXiv,
<https://arxiv.org/html/2508.16688v1>
 49. Combining Agent Memory, Information Retrieval, and Human Knowledge Base -

Karpathy's LLM Wiki - SAP Community,

<https://community.sap.com/t5/artificial-intelligence-blogs-posts/combining-agent-memory-information-retrieval-and-human-knowledge-base/ba-p/14386274>

50. Context engineering in Deep Agents - Docs by LangChain,

<https://docs.langchain.com/oss/javascript/deepagents/context-engineering>